

Containerized Infrastructure and Workload Security

Introduction to containers

Modern systems have 2 problems:
Complexity and **security**.

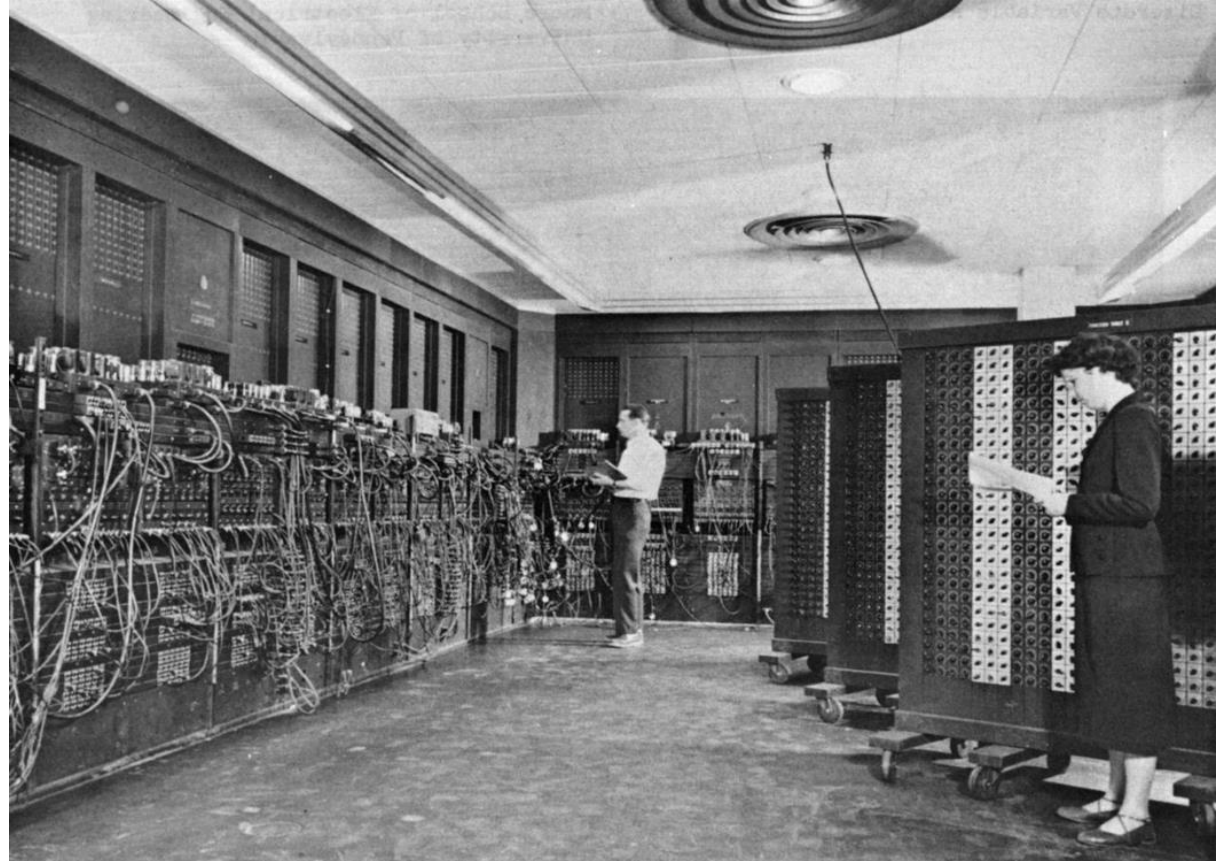
(And the first one is often also the source of the second one.)

– Part 1 –

Complexity

How we got here

- Only 80 years ago (1946), we got the first generally programmable computer (ENIAC)
- There was no software – the program was in the wiring itself
- But it was manageable



15 years later, software became mainstream

(And it has been „eating the world“ ever since)

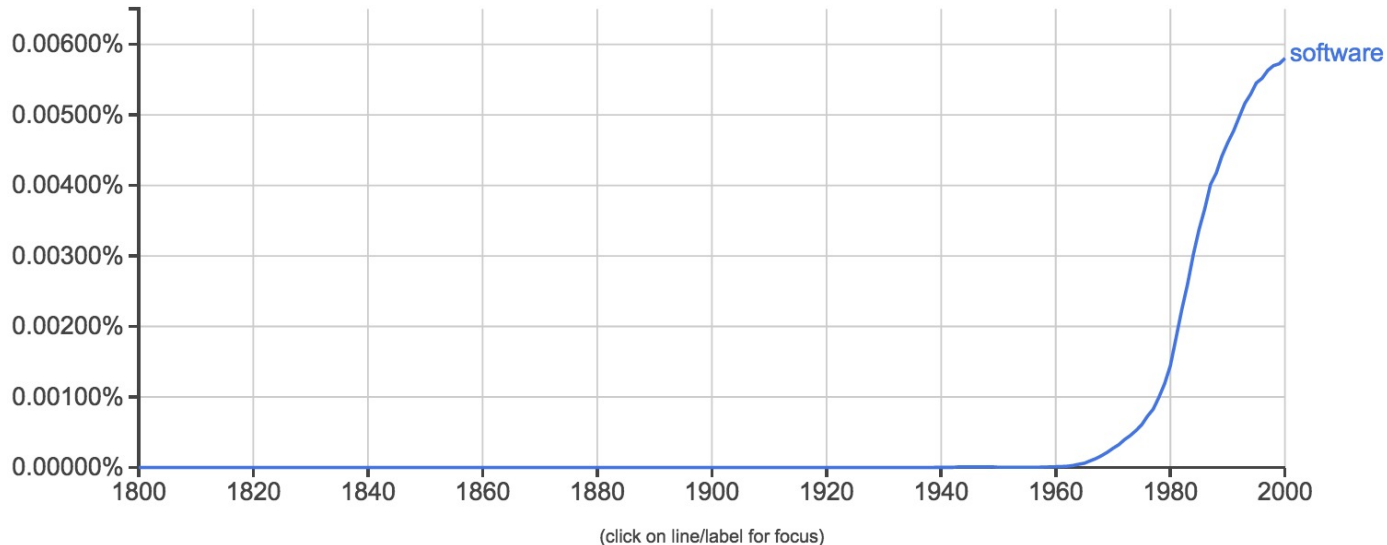
Google books Ngram Viewer

Graph these comma-separated phrases:

☐ case-insensitive

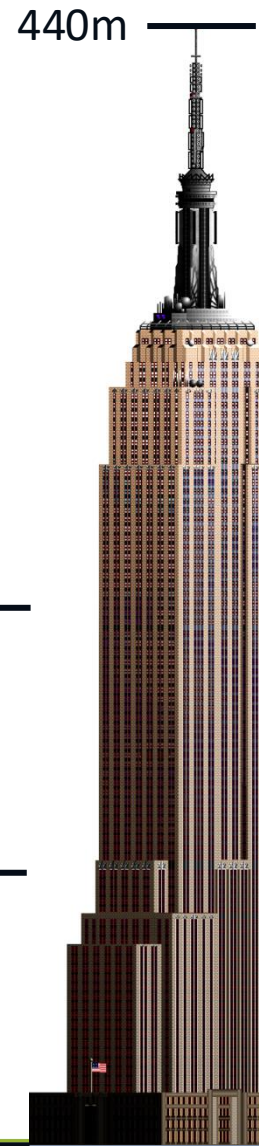
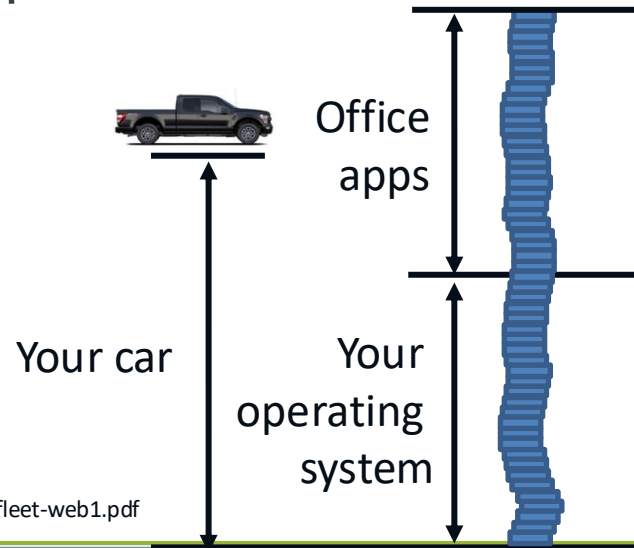
between and from the corpus with smoothing of .

[Search lots of books](#)



Complexity of today's software

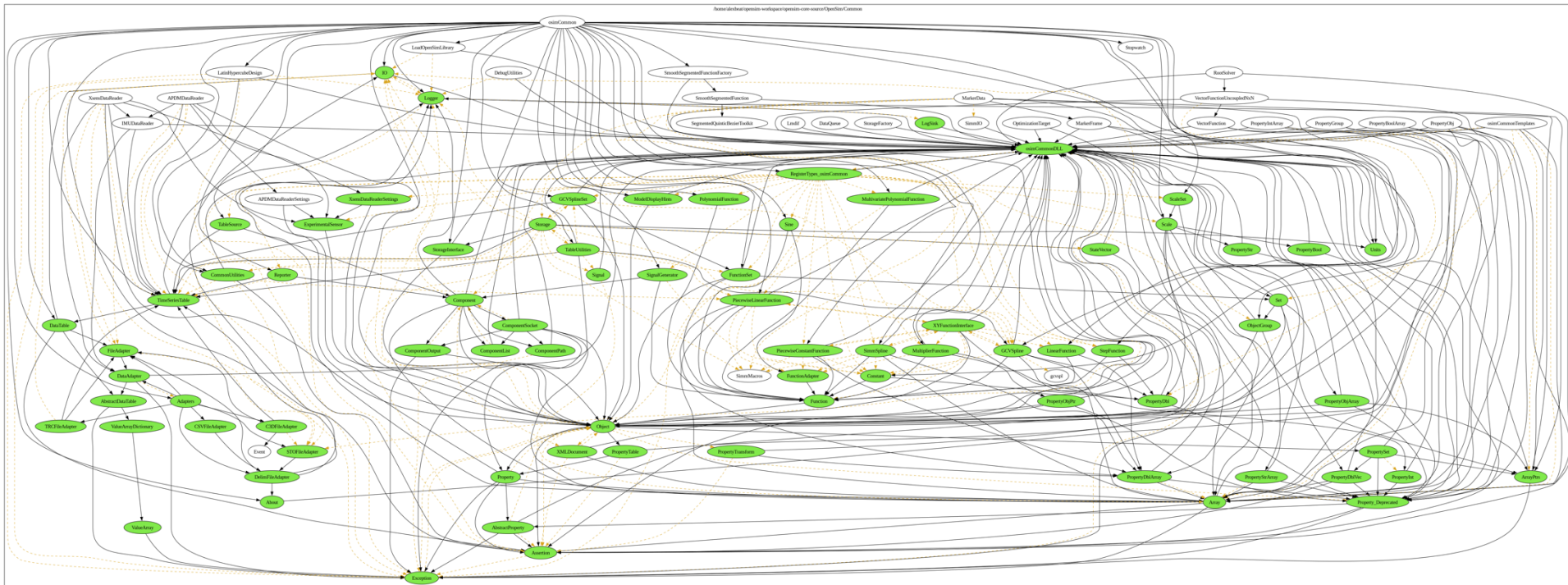
- Consider a book
 - 400 pages = approx 20.000 lines of text
 - Now you know what 20kLOC means
- 1 meter of books $\cong$ 1 million lines of code
- How much code do we use daily?
 - Windows: more than 100M
 - Office: more than 100M
 - Linux kernel: 40M
 - Browser: 21M (Firefox)
 - Average modern car¹: 150M (all subsystems)



¹ source: KPMG <https://advisory.kpmg.us/content/dam/advisory/en/pdfs/protecting-the-fleet-web1.pdf>

Internal dependencies

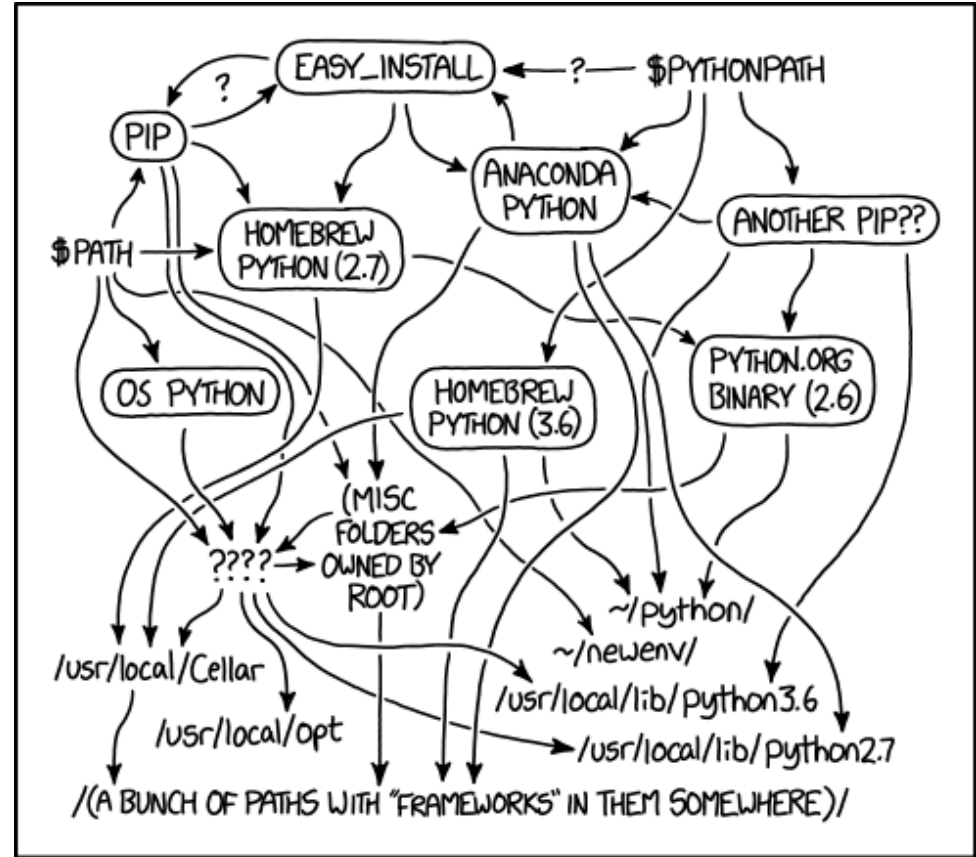
- Modern tools allow us to map internal code structure and dependencies



Source: OpenSim dependency graph

External dependencies → dependency hell

- But there's external dependencies as well
- This is a nightmare to maintain and deploy
- Various possible problems:
 - missing dependency requirements
 - soft-pinned dependencies
 - native (compiled) modules



MY PYTHON ENVIRONMENT HAS BECOME SO DEGRADED
THAT MY LAPTOP HAS BEEN DECLARED A SUPERFUND SITE.

An example horror story

- An open source project from Github
 - Python + some simple imports
- It turns out this project was installable only inside a very narrow 2021 window
 - Numpy 1.19.5 requires python up to 3.6-3.9
 - Python 3.10 came out on October 4, 2021 (not supported)
 - Many dependencies are soft pins:
 - numpy~=1.19.5 means numpy >= 1.19.5, < 1.20.0
 - Package manager will install any available version
 - Several packages have to be built from source
 - numpy, pyarrow, faiss-cpu, grpcio, Pillow, matplotlib
 - And some of these have no source on PyPi
- A lot of archaeology work is needed to determine what works

Requirements.in:

```
# App
facenet-pytorch~=2.5.2
Pillow~=8.1.2
requests~=2.25.1
beautifulsoup4~=4.9.3
numpy~=1.19.5
tqdm~=4.59.0
connexion[swagger-ui]~=2.7.0
environ-config~=20.1.0
flask-cors~=3.0.10
pyarrow~=4.0.1
loguru~=0.5.3
fsspec~=2021.6.0
faiss-cpu~=1.7.1
grpcio~=1.38.0
```

```
# Dev
jupyterlab~=3.0.12
ipykernel~=5.5.0
pip-tools~=6.0.1
pytest~=6.2.2
requests-mock~=1.8.0
pyfakefs~=4.4.0
matplotlib~=3.3.4
cfn-lint~=0.48.2
mock~=4.0.3
pytest-mock~=3.6.0
grpcio-tools~=1.38.0
```





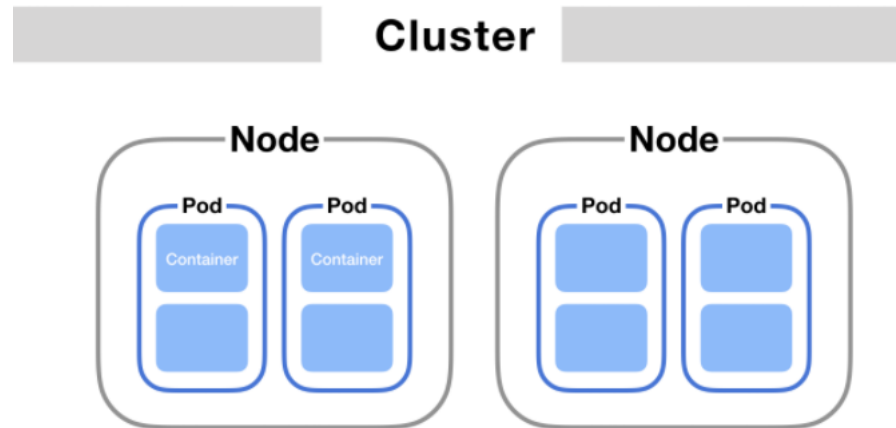


Reproducibility crisis & immutable images

- To ensure any kind of security, we need
 - Reproducible builds → building something twice should yield exactly the same result
- But why wouldn't it?
 - Isn't compilation deterministic?
 - What about dependencies? Bump a minor version → different result
 - Random values? → different results
 - Current timestamp anywhere in the output → different result
- We need ways to "freeze" things down to the last bit

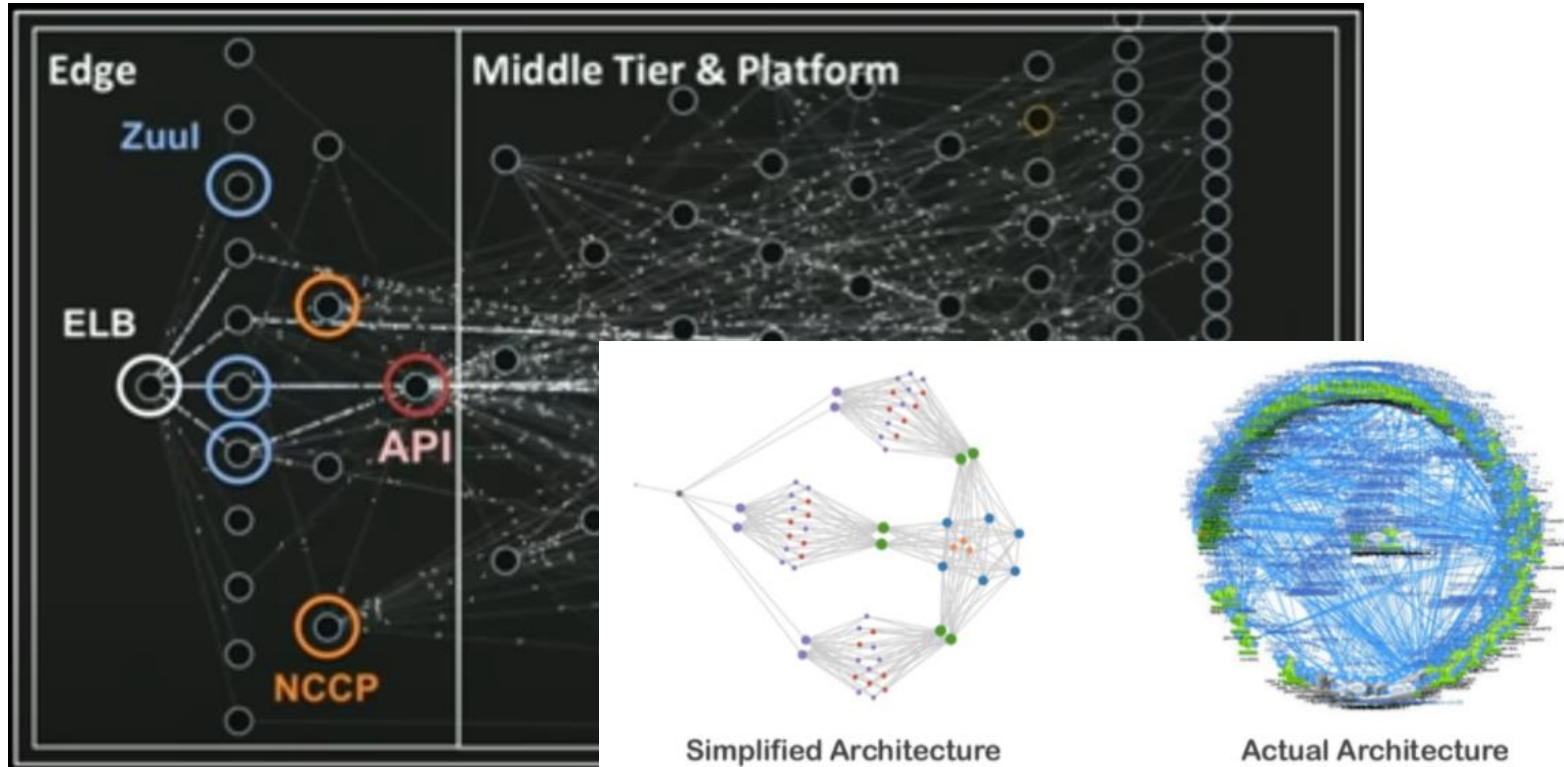
Orchestration and Kubernetes

- And finally, modern systems have 100s of moving parts
- There's no way to manage all of this by hand anymore
- Orchestration platforms such as K8S allow us to automate deployments, upgrades, and scaling



This is Netflix

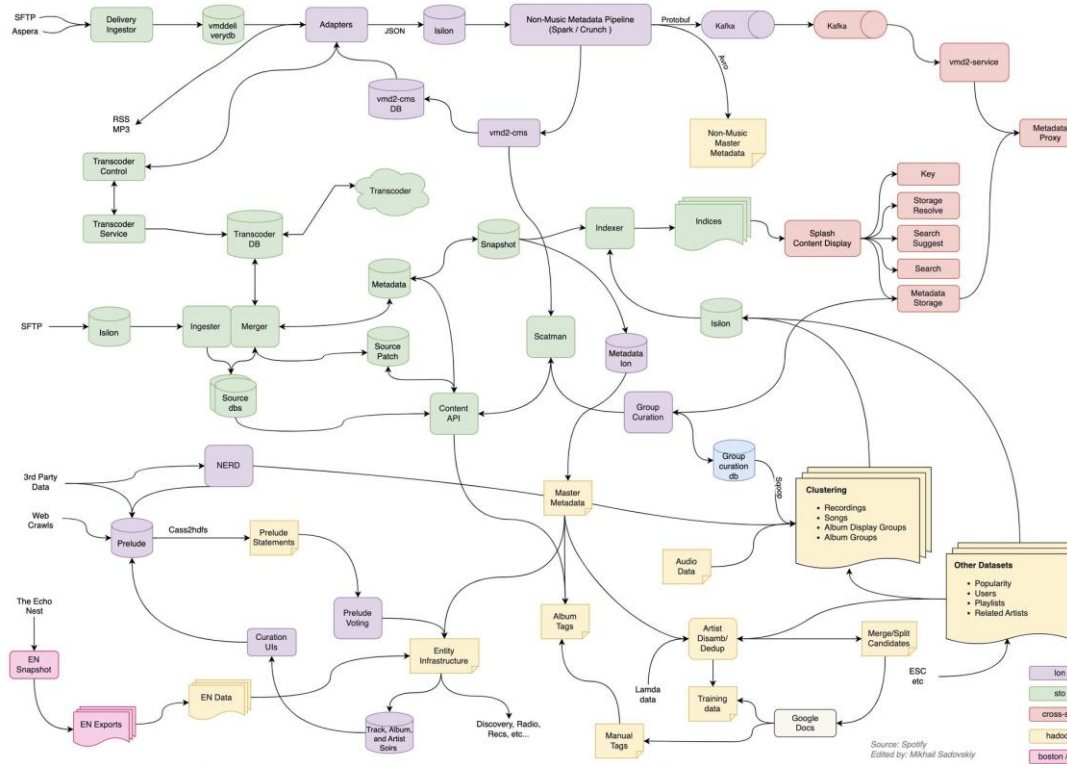
- 700+ microservices



Source: <https://caffeinatedcoder.medium.com/netflixs-microservices-migration-from-monolith-to-700-services-8caa8e5bc574>

This is Spotify

- 810 microservices just to serve some MP3s over HTTP



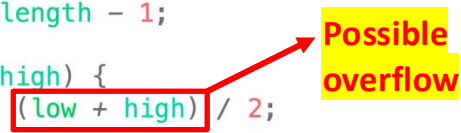
Source: <https://www.slideshare.net/slideshow/microservices-at-spotify/59477153>

– Part 2 – Security

The ability of humans to write correct code

- Famous book: Programming Pearls by Jon Bentley, 1986
 - Bentley was a professor at Carnegie Mellon University
 - World class expert in writing correct and efficient algorithms
 - His book serves as a blueprint for multiple programming languages and libraries (including *java.util.Arrays*)
- Here's one of his algorithms:
 - Binary search, 15 LOC
- 20 years after publication Google found a bug in it
 - Overflow when sorting $>2^{30}$ elements (~a billion)

```
1 public static int binarySearch(int[] a, int item) {
2     int low = 0;
3     int high = a.length - 1;
4
5     while(low <= high) {
6         int mid = (low + high) / 2;
7
8         if(a[mid] == item)
9             return mid;           // Key found
10        else if (a[mid] < item)
11            low = mid + 1;
12        else
13            high = mid - 1;
14    }
15
16    return -(low + 1);           // Key not found
17 }
```



Using 3rd party code (supply chain)

- Practically every application today uses open source libraries / modules
- Once such a module is imported, there should be an audit
- But: will this module update automatically?
- Will every update have a security audit again? – Usually no

Researcher hacks over 35 tech firms in novel supply chain attack

By [Ax Sharma](#)

 February 9, 2021  01:04 PM  2

A researcher managed to breach over 35 major companies' internal systems, including **Microsoft, Apple, PayPal, Shopify, Netflix, Yelp, Tesla, and Uber**, in a novel software supply chain attack.

The attack comprised uploading malware to open source repositories including PyPI, npm, and RubyGems, which then got distributed downstream automatically into the company's internal applications.

Supply chain attack example: XZ utils (2024)

The largest hack that almost happened

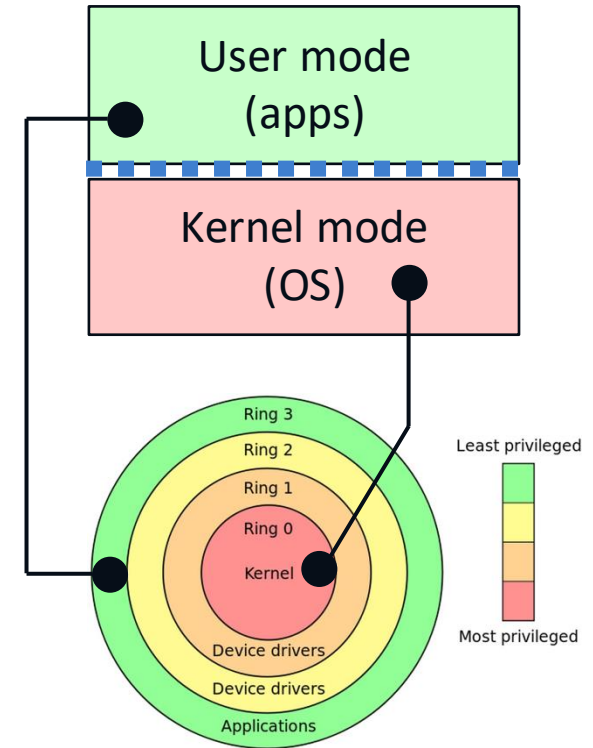


The fundamental problem of security

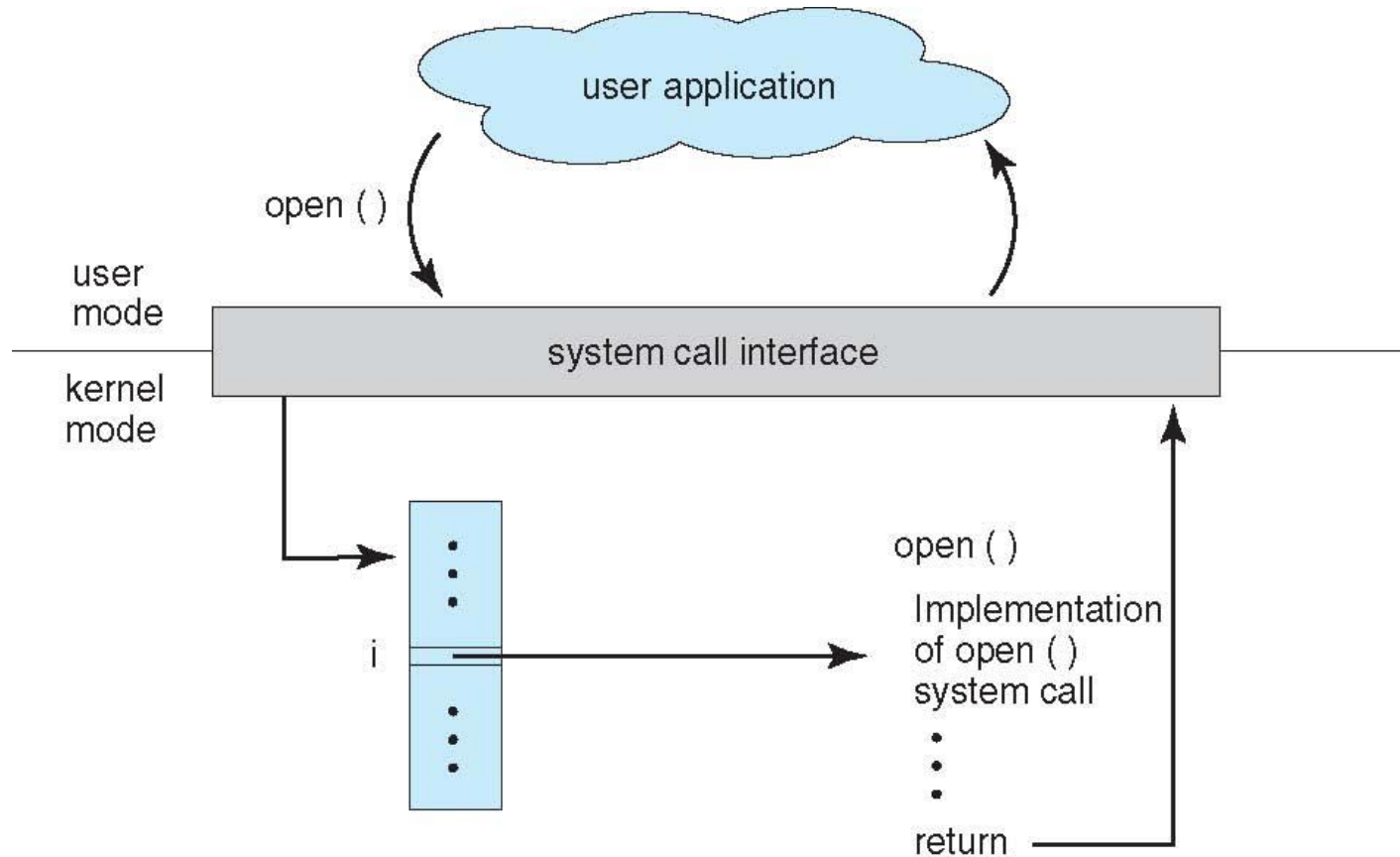
If we can't trust software,
how can we safely run multiple workloads
on one machine?

First attempt: bare metal

- We need an operating system
 - Split it into kernel and user mode
 - This separation is supported by hardware: ring 0 & ring 3 (CPU privilege levels)
 - The interface between them is the syscall API
- The kernel is the security boundary
 - Applications (processes) cannot touch hardware
 - Only kernel can do that
- Processes are somewhat isolated
 - but can still see each other (via syscalls – because kernel allows them to)



System call interface



Linux syscall map

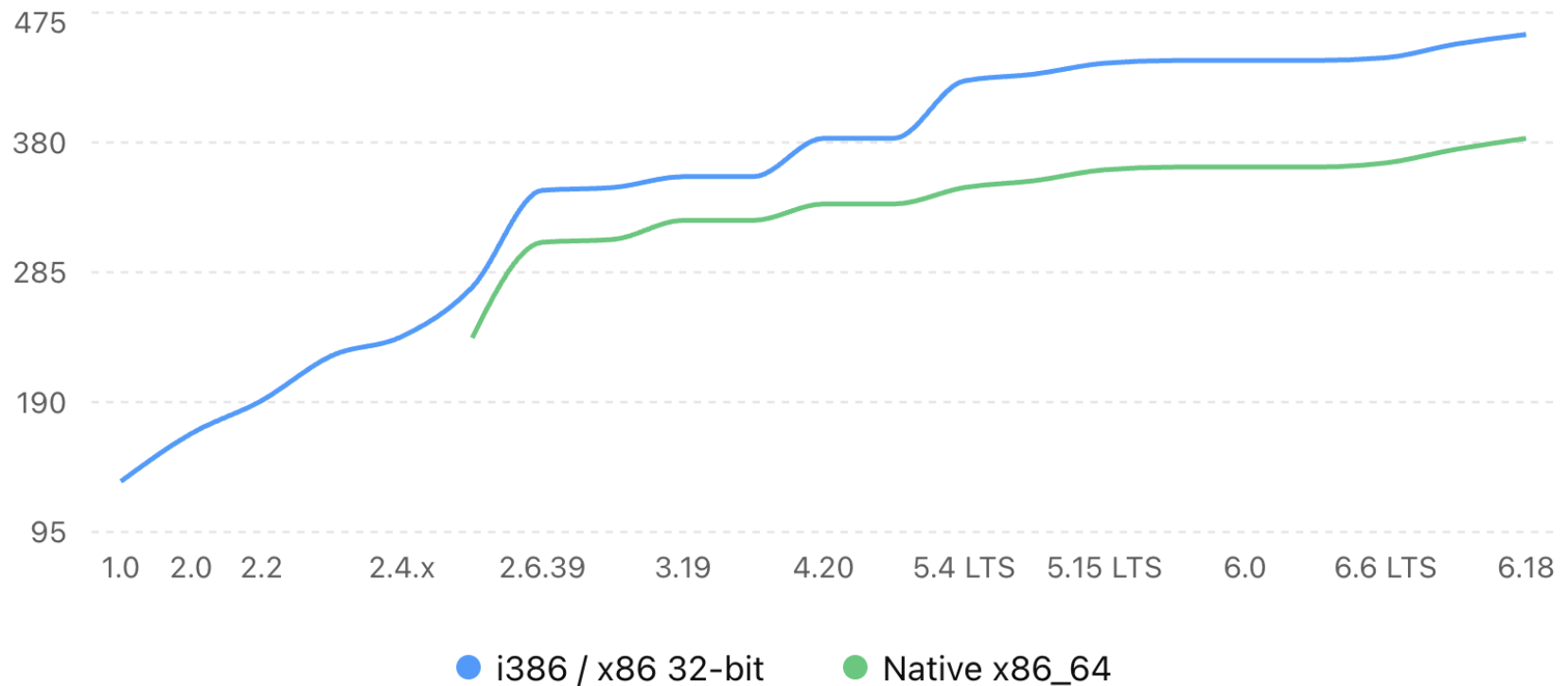
kernel 6.8.0-111-generic

0	read	46	sendmsg	92	chown	138	fstatfs	184	tuxcall	230	clock_nanosleep	276	tee	322	execveat
1	write	47	recvmsg	93	fchown	139	sysfs	185	security	231	exit_group	277	sync_file_range	323	userfaultfd
2	open	48	shutdown	94	lchown	140	getpriority	186	gettid	232	epoll_wait	278	vmsplice	324	membarrier
3	close	49	bind	95	umask	141	setpriority	187	readahead	233	epoll_ctl	279	move_pages	325	mlock2
4	stat	50	listen	96	gettimeofday	142	sched_setparam	188	setxattr	234	tgkill	280	utimensat	326	copy_file_range
5	fstat	51	getsockname	97	getrlimit	143	sched_getparam	189	lsetxattr	235	utimes	281	epoll_pwait	327	preadv2
6	lstat	52	getpeername	98	getrusage	144	sched_setscheduler	190	fsetxattr	236	vserv	282	signalfd	328	pwritev2
7	poll	53	socketpair	99	sysinfo	145	sched_getscheduler	191	getxattr	237	mbind	283	timerfd_create	329	pkey_mprotect
8	lseek	54	setsockopt	100	times	146	sched_get_priority_max	192	lgetxattr	238	set_mempolicy	284	eventfd	330	pkey_alloc
9	mmap	55	getsockopt	101	ptrace	147	sched_get_priority_min	193	fgetxattr	239	get_mempolicy	285	fallocate	331	pkey_free
10	mprotect	56	clone	102	getuid	148	sched_rr_get_interval	194	listxattr	240	mq_open	286	timerfd_settime	332	statx
11	munmap	57	fork	103	syslog	149	mlock	195	llistxattr	241	mq_unlink	287	timerfd_gettime	333	io_pgetevents
12	brk	58	vfork	104	getgid	150	munlock	196	flistxattr	242	mq_timedsend	288	accept4	334	rseq
13	rt_sigaction	59	execve	105	setuid	151	mlockall	197	removexattr	243	mq_timedreceive	289	signalfd4	424	pidfd_send_signal
14	rt_sigprocmask	60	exit	106	setgid	152	munlockall	198	lremovexattr	244	mq_notify	290	eventfd2	425	io_uring_setup
15	rt_sigreturn	61	wait4	107	geteuid	153	vhangup	199	removexattr	245	mq_getsetattr	291	epoll_create1	426	io_uring_enter
16	ioctl	62	kill	108	getegid	154	modify_ldt	200	tkill	246	kexec_load	292	dup3	427	io_uring_register
17	pread	63	uname	109	setpgid	155	pivot_root	201	time	247	waitid	293	pipe2	428	open_tree
18	pwrite	64	semget	110	getppid	156	_sysctl	202	futex	248	add_key	294	inotify_init1	429	move_mount
19	readv	65	semop	111	getpgrp	157	prctl	203	sched_setaffinity	249	request_key	295	preadv	430	fsopen
20	writew	66	semctl	112	setsid	158	arch_prctl	204	sched_getaffinity	250	keyctl	296	pwritev	431	fsconfig
21	access	67	shmdt	113	setreuid	159	adjtimex	205	set_thread_area	251	ioprio_set	297	rt_tgsigqueueinfo	432	fsmount
22	pipe	68	msgget	114	setregid	160	setrlimit	206	io_setup	252	ioprio_get	298	perf_event_open	433	fspick
23	select	69	msgsnd	115	getgroups	161	chroot	207	io_destroy	253	inotify_init	299	recvmsg	434	pidfd_open
24	sched_yield	70	msgrcv	116	setgroups	162	sync	208	io_getevents	254	inotify_add_watch	300	fanotify_init	435	clone3
25	mremap	71	msgctl	117	setresuid	163	acct	209	io_submit	255	inotify_rm_watch	301	fanotify_mark	436	close_range
26	msync	72	fcntl	118	getresuid	164	settimeofday	210	io_cancel	256	migrate_pages	302	prlimit64	437	openat2
27	mincore	73	flock	119	setresgid	165	mount	211	get_thread_area	257	openat	303	name_to_handle_at	438	pidfd_getfd
28	madvise	74	fsync	120	getresgid	166	umount2	212	lookup_dcookie	258	mknodat	304	open_by_handle_at	439	faccessat2
29	shmget	75	fdatasync	121	getpgid	167	swapon	213	epoll_create	259	mknodat	305	clock_adjtime	440	process_madvise
30	shmat	76	truncate	122	setfsuid	168	swapoff	214	epoll_ctl_old	260	fchownat	306	syncfs	441	epoll_pwait2
31	shmctl	77	ftruncate	123	setfsuid	169	reboot	215	epoll_wait_old	261	futimesat	307	sendmmsg	442	mount_setattr
32	dup	78	getdents	124	getsid	170	sethostname	216	remap_file_pages	262	newfstatat	308	setns	443	quotactl_fd
33	dup2	79	getcwd	125	capget	171	setdomainname	217	getdents64	263	unlinkat	309	getcpu	444	landlock_create_ruleset
34	pause	80	chdir	126	capset	172	iopl	218	set_tid_address	264	renameat	310	process_vm_readv	445	landlock_add_rule
35	nanosleep	81	fchdir	127	rt_sigpending	173	ioperm	219	restart_syscall	265	linkat	311	process_vm_writev	446	landlock_restrict_self
36	getitimer	82	rename	128	rt_sigtimedwait	174	create_module	220	semtimedop	266	symlinkat	312	kcmp	447	memfd_secret
37	alarm	83	mkdir	129	rt_sigqueueinfo	175	init_module	221	fadvise64	267	readlinkat	313	finit_module	448	process_mrelease
38	setitimer	84	rmdir	130	rt_sigsuspend	176	delete_module	222	timer_create	268	fchmodat	314	sched_setattr	449	futex_waitv
39	getpid	85	creat	131	sigaltstack	177	get_kernel_syms	223	timer_settime	269	faccessat	315	sched_getattr	450	set_mempolicy_home_node
40	sendfile	86	link	132	utime	178	query_module	224	timer_gettime	270	pselect6	316	renameat2	451	cachestat
41	socket	87	unlink	133	mknod	179	quotactl	225	timer_getoverrun	271	ppoll	317	seccomp		
42	connect	88	symlink	134	uselib	180	nfsservctl	226	timer_delete	272	unshare	318	getrand		
43	accept	89	readlink	135	personality	181	getpmsg	227	clock_settime	273	set_robust_list	319	memfd_create		
44	sendto	90	chmod	136	ustat	182	putpmsg	228	clock_gettime	274	get_robust_list	320	kexec_file_load		
45	recvfrom	91	fchmod	137	statfs	183	afs_syscall	229	clock_getres	275	splice	321	bpf		

Number of Linux kernel syscalls through time

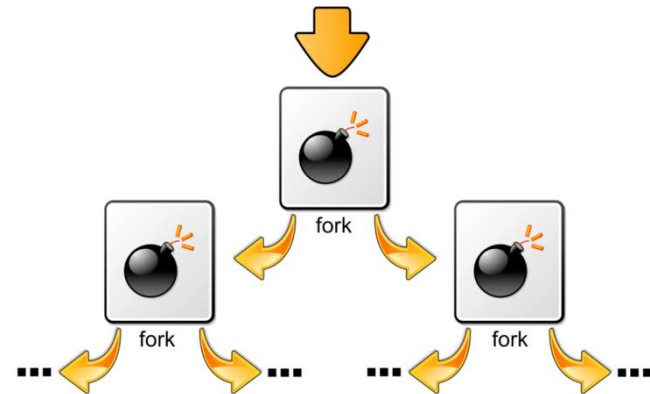
Important fact: the backwards compatibility is extremely good

- Linux treats an exposed syscall interface as effectively permanent



Some things a process can do

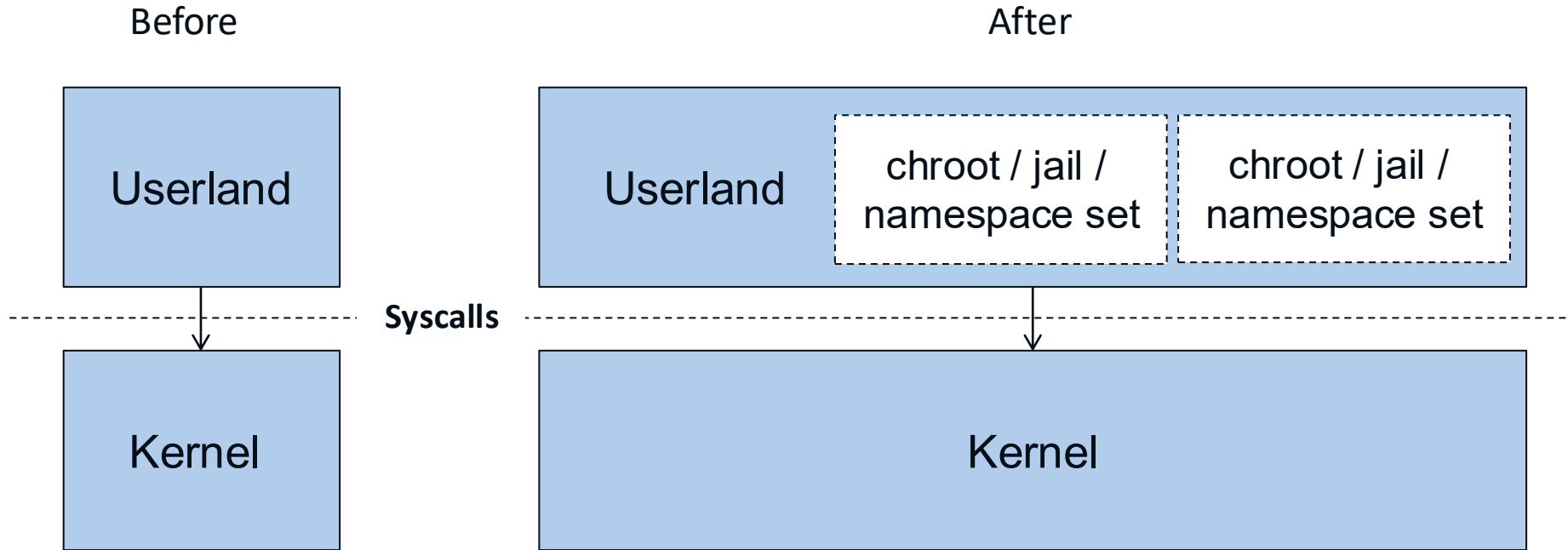
- Obviously – it can't touch hardware directly
 - Needs to ask the kernel for that (via system calls)
- But kernel will happily let it:
 - wipe any data it can write
 - leak any secrets it can read
 - inspect other host processes
 - kill same-user processes
 - fork-bomb the machine
 - exhaust CPU / memory / PIDs
 - bind to host ports above 1024
 - fill the disk



Emergence of better isolation mechanisms

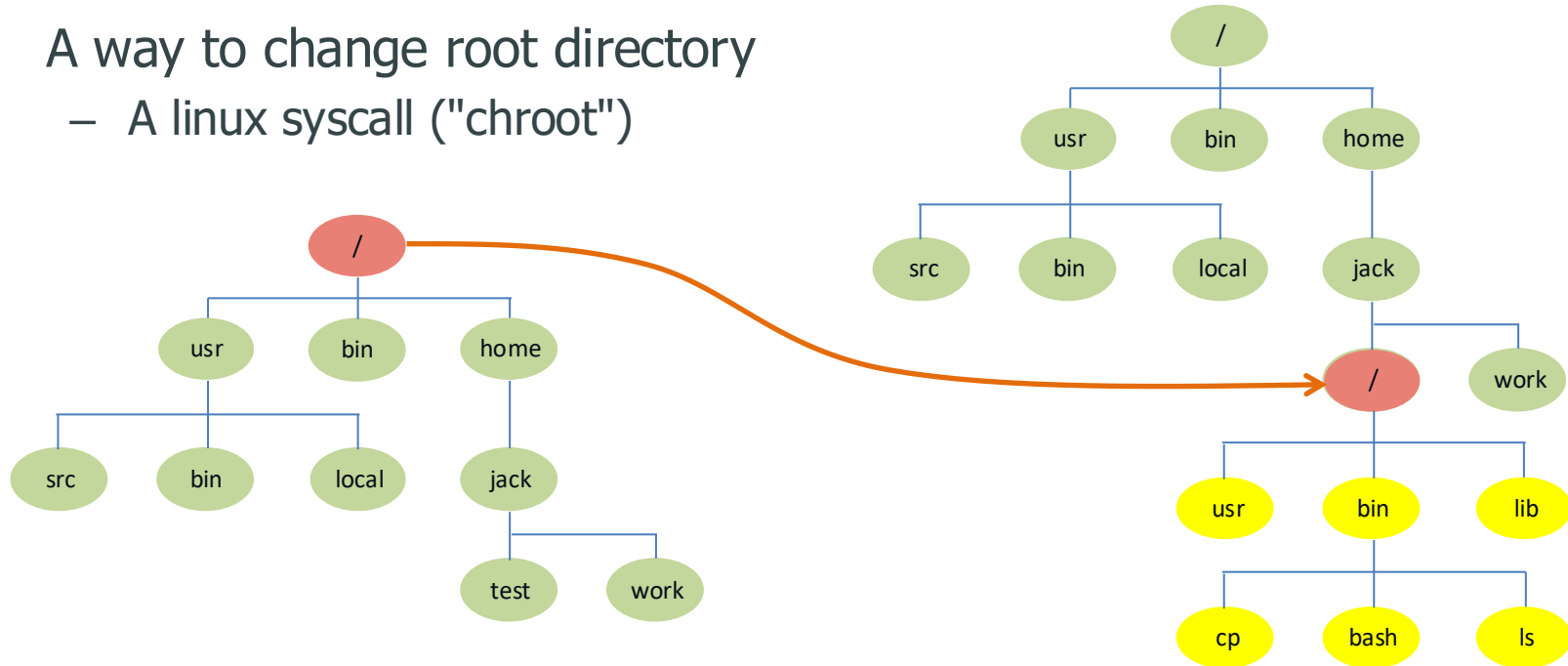
- What if processes should not even *see* the host
 - filesystem,
 - process tree
 - network stack...?
- This has led to stronger isolation mechanisms within the same OS
 - chroot (1979) – change what the process can see as / (=fake root)
 - BSD jails (2000) – isolation of file system, process tree, users, networking
 - Linux Virtuozzo / OpenVZ (2002/05) – isolation of FS, process tree, users, networking
 - (The last two are what we now call „system containers“)
- Ancestors of modern containers
- Still one (shared) kernel
 - But stronger process isolation

Shared kernel, isolated userland environments



Illustrative example: what is chroot

- A way to change root directory
 - A linux syscall ("chroot")



- `sudo chroot /home/jack/test /bin/bash`

How is chroot implemented?

- First appeared in UNIX v7

- <https://github.com/v7unix/v7unix/blob/master/v7/usr/sys/sys/sys4.c>

```
194     chdir()
195     {
196         chdirec(&u.u_cdir);
197     }
198
199     chroot()
200     {
201         if (suser())
202             chdirec(&u.u_rdir);
203     }
```



It basically just changes the "root dir" parameter of a process

The process cannot "cd .." out of what it sees as its own root

- <https://github.com/v7unix/v7unix/blob/master/v7/usr/sys/h/user.h>

```
45     struct inode *u_cdir;        /* pointer to inode of current directory */
46     struct inode *u_rdir;        /* root directory of current process */
```

Linux syscall map

0	read	46	sendmsg	92	chown	138	fstatfs	184	tuxcall	230	clock_nanosleep	276	tee	322	execveat
1	write	47	recvmsg	93	fchown	139	sysfs	185	security	231	exit_group	277	sync_file_range	323	userfaultfd
2	open	48	shutdown	94	lchown	140	getpriority	186	gettid	232	epoll_wait	278	vmsplice	324	membarrier
3	close	49	bind	95	umask	141	setpriority	187	readahead	233	epoll_ctl	279	move_pages	325	mlock2
4	stat	50	listen	96	gettimeofday	142	sched_setparam	188	setxattr	234	tgkill	280	utimensat	326	copy_file_range
5	fstat	51	getsockname	97	getrlimit	143	sched_getparam	189	lsetxattr	235	utimes	281	epoll_pwait	327	preadv2
6	lstat	52	getpeername	98	getrusage	144	sched_setscheduler	190	fsetxattr	236	vserv	282	signalfd	328	pwritev2
7	poll	53	socketpair	99	sysinfo	145	sched_getscheduler	191	getxattr	237	mbind	283	timerfd_create	329	pkey_mprotect
8	lseek	54	setsockopt	100	times	146	sched_get_priority_max	192	lgetxattr	238	set_mempolicy	284	eventfd	330	pkey_alloc
9	mmap	55	getsockopt	101	ptrace	147	sched_get_priority_min	193	fgetxattr	239	get_mempolicy	285	fallocate	331	pkey_free
10	mprotect	56	clone	102	getuid	148	sched_rr_get_interval	194	listxattr	240	mq_open	286	timerfd_settime	332	statx
11	munmap	57	fork	103	syslog	149	mlock	195	llistxattr	241	mq_unlink	287	timerfd_gettime	333	io_pgetevents
12	brk	58	vfork	104	getgid	150	munlock	196	flistxattr	242	mq_timedsend	288	accept4	334	rseq
13	rt_sigaction	59	execve	105	setuid	151	mlockall	197	removexattr	243	mq_timedreceive	289	signalfd4	424	pidfd_send_signal
14	rt_sigprocmask	60	exit	106	setgid	152	mlockall	198	lremovexattr	244	mq_notify	290	eventfd2	425	io_uring_setup
15	rt_sigreturn	61	wait4	107	geteuid	153	vhangup	199	fremovexattr	245	mq_getsetattr	291	epoll_create1	426	io_uring_enter
16	ioctl	62	kill	108	getegid	154	modify_ldt	200	tkill	246	kexec_load	292	dup3	427	io_uring_register
17	pread	63	uname	109	setpgid	155	pivot_root	201	time	247	waitid	293	pipe2	428	open_tree
18	pwrite	64	semget	110	getppid	156	_sysctl	202	futex	248	add_key	294	inotify_init1	429	move_mount
19	readv	65	semop	111	getpgrp	157	prctl	203	sched_setaffinity	249	request_key	295	preadv	430	fsopen
20	writew	66	semctl	112	setsid	158	arch_prctl	204	sched_getaffinity	250	keyctl	296	pwritev	431	fsconfig
21	access	67	shmdt	113	setreuid	159	adjtimex	205	set_thread_area	251	ioprio_set	297	rt_tsigqueueinfo	432	fsmount
22	pipe	68	msgget	114	setregid	160	setrlimit	206	io_setup	252	ioprio_get	298	perf_event_open	433	fspick
23	select	69	msgsnd	115	getgroups	161	chroot	207	io_destroy	253	inotify_init	299	recvmsg	434	pidfd_open
24	sched_yield	70	msgrcv	116	setgroups	162	sync	208	io_getevents	254	inotify_add_watch	300	fanotify_init	435	clone3
25	mremap	71	msgctl	117	setresuid	163	acct	209	io_submit	255	inotify_rm_watch	301	fanotify_mark	436	close_range
26	msync	72	fcntl	118	getresuid	164	settimeofday	210	io_cancel	256	migrate_pages	302	prlimit64	437	openat2
27	mincore	73	flock	119	setresgid	165	mount	211	get_thread_area	257	openat	303	name_to_handle_at	438	pidfd_getfd
28	madvise	74	fsync	120	getresgid	166	umount2	212	lookup_dcookie	258	mknodat	304	open_by_handle_at	439	faccessat2
29	shmget	75	fdatasync	121	getpgid	167	swapon	213	epoll_create	259	mknodat	305	clock_adjtime	440	process_madvise
30	shmat	76	truncate	122	setfsuid	168	swapoff	214	epoll_ctl_old	260	fchownat	306	syncfs	441	epoll_pwait2
31	shmctl	77	ftruncate	123	setfsuid	169	reboot	215	epoll_wait_old	261	futimesat	307	sendmmsg	442	mount_setattr
32	dup	78	getdents	124	getfsid	170	sethostname	216	remap_file_pages	262	newfstatat	308	setns	443	quotactl_fd
33	dup2	79	getcwd	125	capget	171	setdomainname	217	getdents64	263	unlinkat	309	getcpu	444	landlock_create_ruleset
34	pause	80	chdir	126	capset	172	iopl	218	set_tid_address	264	renameat	310	process_vm_readv	445	landlock_add_rule
35	nanosleep	81	fchdir	127	rt_sigpending	173	ioperm	219	restart_syscall	265	linkat	311	process_vm_writev	446	landlock_restrict_self
36	getitimer	82	rename	128	rt_sigtimedwait	174	create_module	220	semtimedop	266	symlinkat	312	kcmp	447	memfd_secret
37	alarm	83	mknod	129	rt_sigqueueinfo	175	init_module	221	fadvise64	267	readlinkat	313	finit_module	448	process_mrelease
38	setitimer	84	rmdir	130	rt_sigsuspend	176	delete_module	222	timer_create	268	fchmodat	314	sched_setattr	449	futex_waitv
39	getpid	85	creat	131	sigaltstack	177	get_kernel_syms	223	timer_settime	269	faccessat	315	sched_getattr	450	set_mempolicy_home_node
40	sendfile	86	link	132	utime	178	query_module	224	timer_gettime	270	pselect6	316	renameat2	451	cachestat
41	socket	87	unlink	133	mknod	179	quotactl	225	timer_getoverrun	271	ppoll	317	seccomp		
42	connect	88	symlink	134	uselib	180	nfservctl	226	timer_delete	272	unshare	318	getrandom		
43	accept	89	readlink	135	personality	181	getpmsg	227	clock_settime	273	set_robust_list	319	memfd_create		
44	sendto	90	chmod	136	ustat	182	putpmsg	228	clock_gettime	274	get_robust_list	320	kexec_file_load		
45	recvfrom	91	fchmod	137	statfs	183	afs_syscall	229	clock_getres	275	splice	321	bpf		

But the kernel is still shared

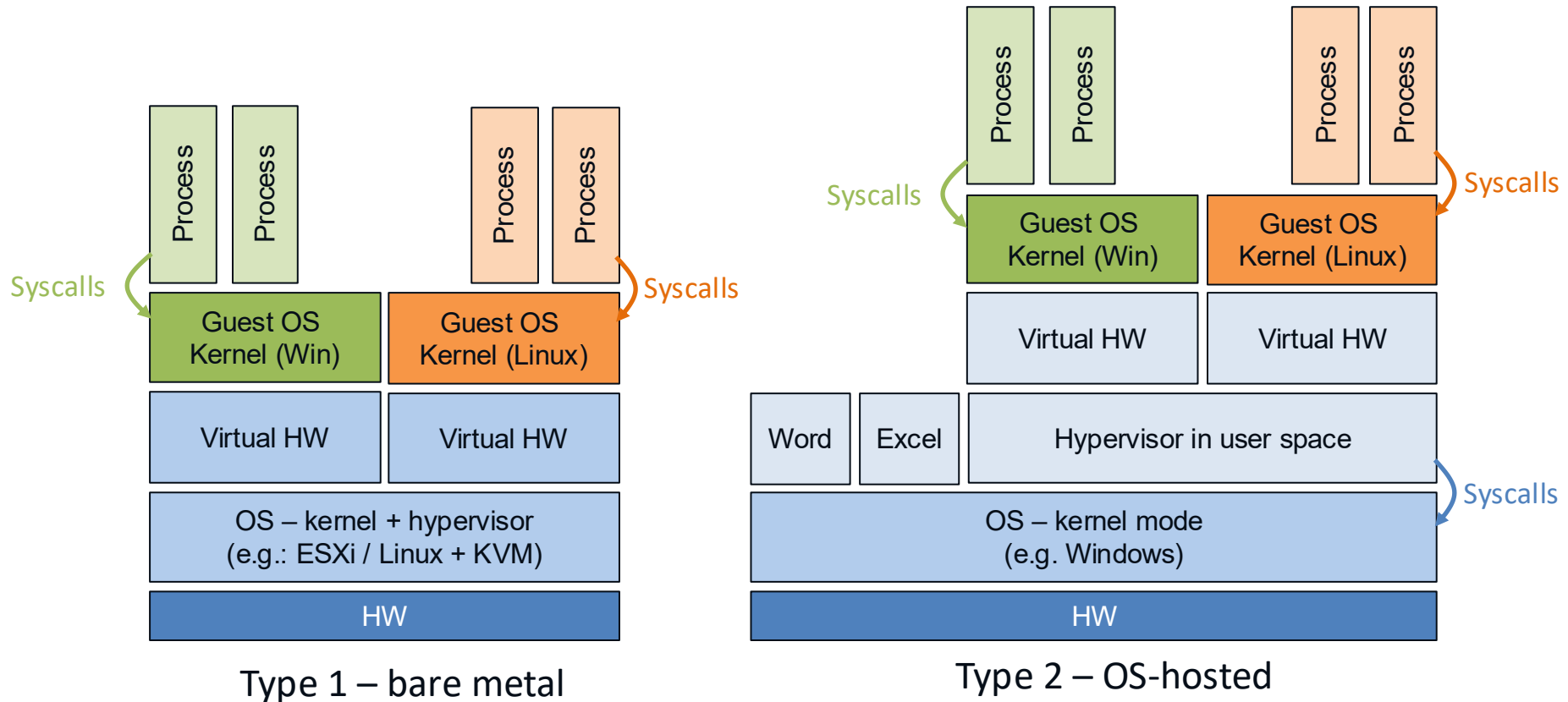
- Not only a problem for security, but also reliability

```
[ 1.076702] Kernel panic - not syncing: UFS: Unable to mount root fs on unknow-
n-block(0,0)
[ 1.077718] CPU: 0 PID: 1 Comm: swapper/0 Not tainted 3.10.0-327.el7.x86_64 #
1
[ 1.078657] Hardware name: VMware, Inc. VMware Virtual Platform/440BX Desktop
Reference Platform, BIOS 6.00 07/31/2013
[ 1.079594] ffffffff8184e928 000000001e6559f5 ffff880139387d60 ffffffff81635
1f1
[ 1.080528] ffff880139387de0 ffffffff8162ea6c ffffffff00000010 ffff880139387
df0
[ 1.081446] ffff880139387d90 000000001e6559f5 000000001e6559f5 ffff880139387
e00
[ 1.082371] Call Trace:
[ 1.082616] [<ffffffff816351f1>] dump_stack+0x19/0x1b
[ 1.083005] [<ffffffff8162ea6c>] panic+0xd8/0x1e7
[ 1.083382] [<ffffffff81a8d5fa>] mount_block_root+0x2a1/0x2b0
[ 1.083826] [<ffffffff81a8d65c>] mount_root+0x53/0x56
[ 1.084223] [<ffffffff81a8d79b>] prepare_namespace+0x13c/0x174
[ 1.084667] [<ffffffff81a8d268>] kernel_init_freeable+0x1f0/0x217
[ 1.085125] [<ffffffff81a8c9db>] ? initcall_blacklist+0xb0/0xb0
[ 1.085570] [<ffffffff81624e10>] ? rest_init+0x80/0x80
[ 1.085961] [<ffffffff81624e1e>] kernel_init+0xe/0xf0
[ 1.087300] [<ffffffff81645858>] ret_from_fork+0x58/0x90
```

What if we want real isolation?

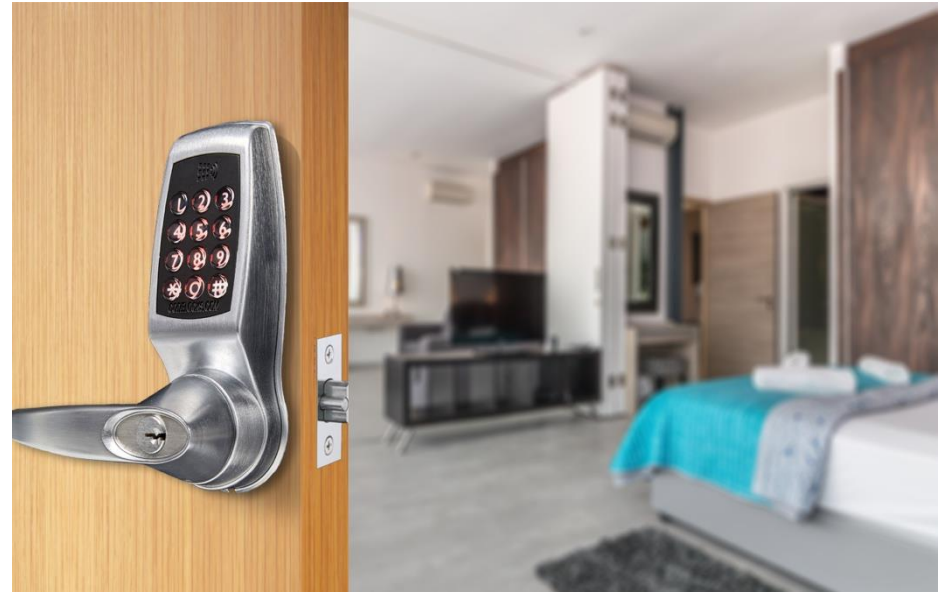
- Virtualization is our best option
 - Hypervisor is the “virtual machine monitor”
 - It divides the HW resources and presents multiple virtual machines
 - Hardware assistance is needed to make this efficient (VT-X or AMD-V)
 - Each guest OS thinks it's running on its own machine
- Full virtualization is not emulation
 - Virtual machines run on the real (host) CPU and talk to real (host) memory
- But: virtualization needs to emulate most of the peripheral devices
 - Paravirtualization & virtIO define APIs where guest and hypervisor can stop pretending these devices are real (API instead of HW emulation)
- Two main kinds of full virtualization
 - Type 1: bare metal: hypervisor is part of the kernel itself – talks to HW
 - Type 2: OS-level: hypervisor is just an app that needs to make syscalls

Type 1 vs. Type 2 virtualization



Container vs. VM

- Metaphor: couchsurfer vs. AirBnB
 - Isolation is much poorer if someone's already in the apartment



If VMs are so much more secure,
why are containers so popular?

Containers vs. VMs

- Key differences: containers are smaller and have less overhead
 - Faster deployments enable more powerful automation

	Container	VM
Managed by	OS kernel	Hypervisor
Kernel is	Shared with the host	Has its own kernel
Startup time	Seconds	Minutes
Image size	MBs	GBs
Replacability	Low (system) or high (app)	Low
Isolation	Lower	Higher
Overhead	2-3x smaller	
Automation	Easy	Hard

Linux containers

- Linux containers are just „chroot on steroids“
 - Isolated processes build on top of basic kernel features
- Two most important features
 - Namespaces
 - first namespace appeared in 2002
 - Control groups
 - since 2007
- + some others we'll touch upon later

The main two mechanisms of isolation

- **Namespaces**

- Partition resources into "buckets";
- Processes in one bucket have no visibility into other buckets
- There's many kinds of namespaces
 - Mount namespaces (2002)
 - IPC namespace (2006): shared memory identifiers
 - PID namespaces (2008): real PIDs are mapped to 1...N inside a namespace
 - Network namespaces (2009): separate interface and network stack per container
 - User namespaces (2013): remapped users and groups (root inside = non-root outside)
 - Cgroup namespaces (2016): limit control group visibility
 - Time namespaces (2020): boot time per namespace

- **Control groups (cgroups)**

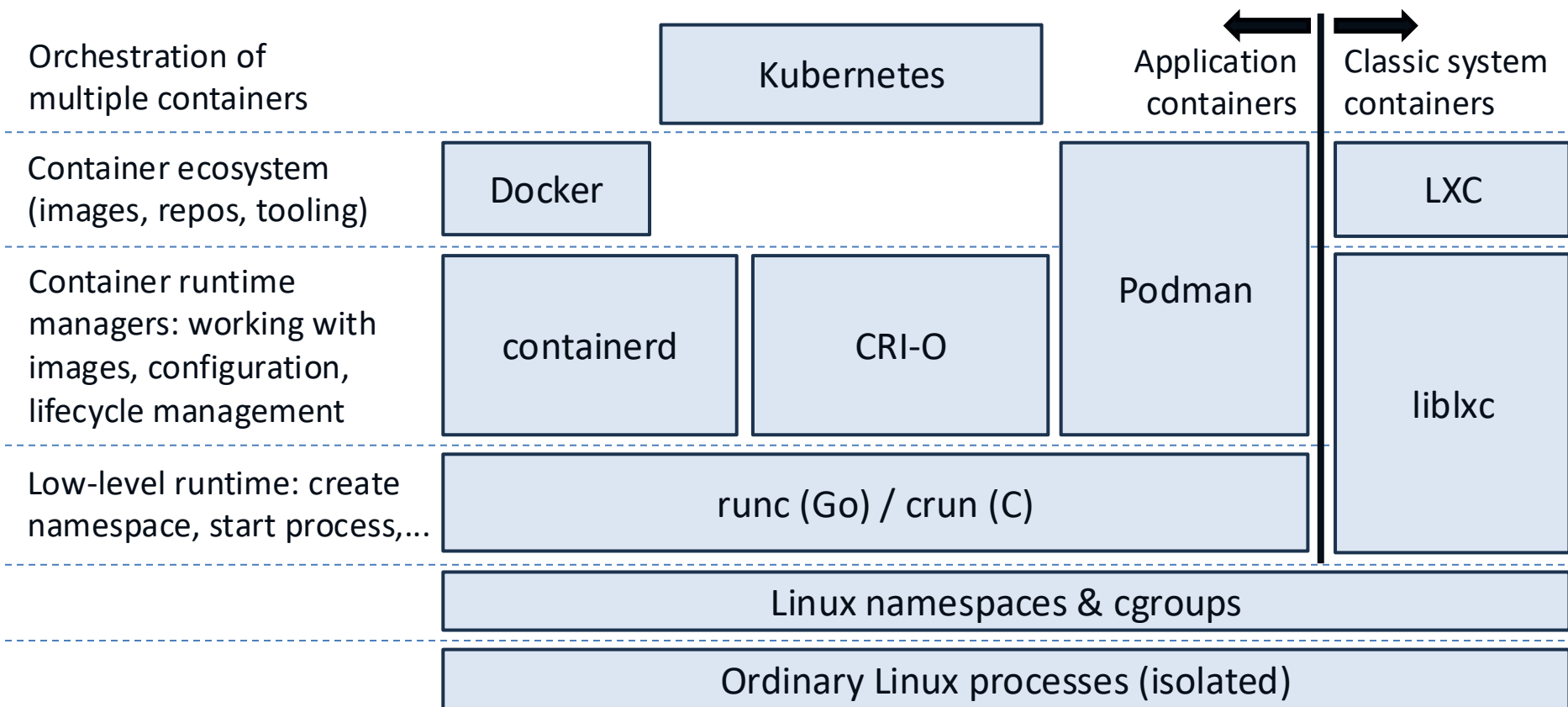
- Limiting how much resources a process can consume
 - Memory limit, CPU quota limit, I/O bandwidth limit, maximum open files, etc.
- By default, scheduling on Linux allows only prioritization, not limiting of resources

Some things a containerized process can do

- Obviously – it can't touch hardware directly
 - Needs to ask the kernel for that (via system calls)
- But kernel will happily let it:
 - ~~wipe any data it can write~~ → only in its mount namespace
 - ~~leak any secrets it can read~~ → only in its mount namespace
 - ~~inspect other host processes~~ → only in its PID namespace
 - ~~kill same user processes~~ → only in its PID namespace
 - ~~fork bomb the machine~~ → capped by the pids cgroup
 - ~~exhaust CPU / memory / PIDs~~ → capped by cgroup limits
 - ~~bind to host ports above 1024~~ → only in its network namespace
 - ~~fill the disk~~ → capped by quotas on the underlying storage

Everything is just Linux kernel under the hood

- Docker, Podman, LXC, ... Everything becomes "kernel primitives"



Docker



- Most popular container ecosystem (by far)
 - But it's just some *tooling*
 - And an "app store" – Docker Hub



CLI Cheat Sheet

Docker provides the ability to package and run an application in a loosely isolated environment called a container. The isolation and security allows you to run many containers simultaneously on a given host. Containers are lightweight and contain everything needed to run the application, so you do not need to rely on what is currently installed on the host. You can easily share containers while you work, and be sure that everyone you share with gets the same container that works in the same way.

INSTALLATION

Docker Desktop is available for Mac, Linux and Windows
<https://docs.docker.com/desktop>

View example projects that use Docker
<https://github.com/docker/awesome-compose>

Check out our docs for information on using Docker
<https://docs.docker.com>

GENERAL COMMANDS

Start the docker daemon
`docker -d`

Get help with Docker. Can also use `--help` on all subcommands
`docker --help`

Display system-wide information
`docker info`

IMAGES

Docker images are a lightweight, standalone, executable package of software that includes everything needed to run an application: code, runtime, system tools, system libraries and settings.

Build an image from a Dockerfile
`docker build -t <image_name> .`

Build an image from a Dockerfile without the cache
`docker build -t <image_name> . --no-cache`

List local images
`docker images`

Delete an image
`docker rmi <image_name>`

Remove all unused images
`docker image prune`

CONTAINERS

A container is a runtime instance of a docker image. A container will always run the same, regardless of the infrastructure. Containers isolate software from its environment and ensure that it works uniformly despite differences for instance between development and staging.

Create and run a container from an image, with a custom name:
`docker run --name <container_name> <image_name>`

Run a container with and publish a container's port(s) to the host:
`docker run -p <host_port>:<container_port> <image_name>`

Run a container in the background
`docker run -d <image_name>`

Start or stop an existing container:
`docker start|stop <container_name> (or <container_id>)`

Remove a stopped container:
`docker rm <container_name>`

Open a shell inside a running container:
`docker exec -it <container_name> sh`

Fetch and follow the logs of a container:
`docker logs -f <container_name>`

To inspect a running container:
`docker inspect <container_name> (or <container_id>)`

To list currently running containers:
`docker ps`

List all docker containers (running and stopped):
`docker ps --all`

View resource usage stats
`docker container stats`

DOCKER HUB

Docker Hub is a service provided by Docker for finding and sharing container images with your team. Learn more and find images at <https://hub.docker.com>

Login into Docker
`docker login -u <username>`

Publish an image to Docker Hub
`docker push <username>/<image_name>`

Search Hub for an image
`docker search <image_name>`

Pull an image from a Docker Hub
`docker pull <image_name>`

Case in point: Bocker

Bocker

Docker implemented in around 100 lines of bash.

- [Prerequisites](#)
- [Example Usage](#)
- [Functionality: Currently Implemented](#)
- [Functionality: Not Yet Implemented](#)
- [License](#)

Prerequisites

The following packages are needed to run bocker.

- btrfs-progs
- curl
- iproute2
- iptables
- libcgrouptools
- util-linux >= 2.25.2
- coreutils >= 7.5

Example Usage

```
$ bocker pull centos 7
##### 100.0%
##### 100.0%
##### 100.0%
Created: img_42150

$ bocker images
IMAGE_ID      SOURCE
img_42150     centos:7

$ bocker run img_42150 cat /etc/centos-release
CentOS Linux release 7.1.1503 (Core)

$ bocker ps
CONTAINER_ID  COMMAND
ps_42045      cat /etc/centos-release

$ bocker logs ps_42045
CentOS Linux release 7.1.1503 (Core)

$ bocker rm ps_42045
Removed: ps_42045

$ bocker run img_42150 which wget
which: no wget in (/usr/local/sbin:/usr/local/bin:/usr/sbin:/usr/bin:/root/bin)
```

Source: <https://github.com/p8952/bocker>

System vs. application containers

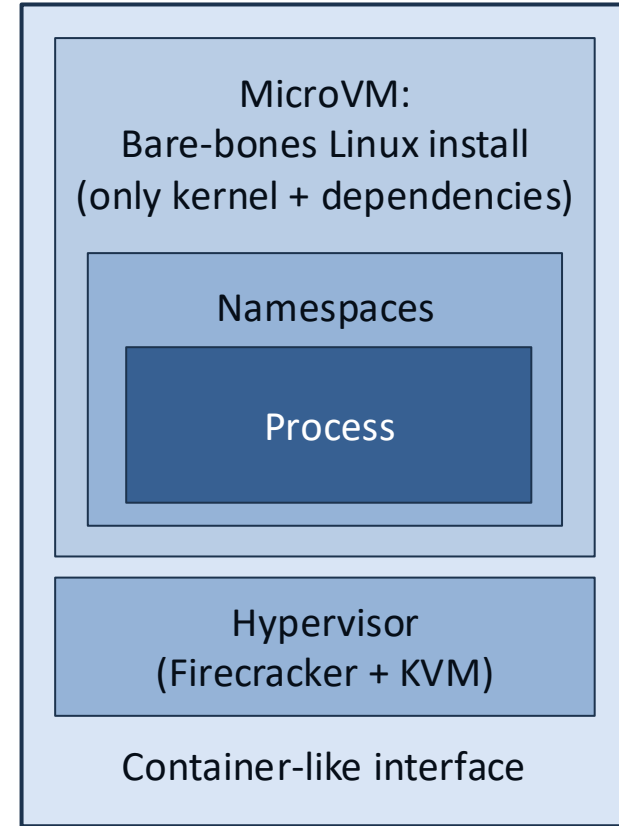
- The difference is more philosophical than technical
- System containers appear to be whole systems
 - Older of the two philosophies (2005 for OpenVZ, 2008 for LXC)
 - They encourage running multiple processes, which mimics a whole system
 - Tech stacks actually diverge (different image formats, different tooling)
- Application containers focus on running only 1 app/service each
 - This often means only 1 "useful" process per container
 - Designed to be replaceable / disposable (all state should be stored outside)
 - Easier health monitoring (can be only up or down)
 - Easier log collection (everything in stdout is one log stream)
 - Better fit for the microservices paradigm
 - More straightforward scaling (scale services independently) – more compatible with orchestration

The security realization

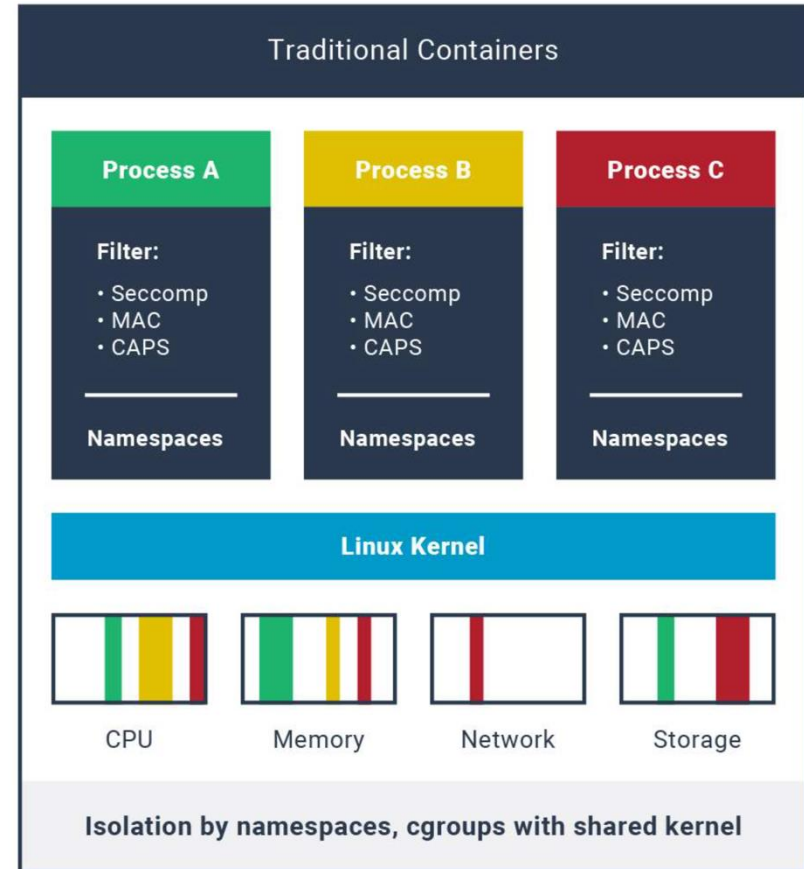
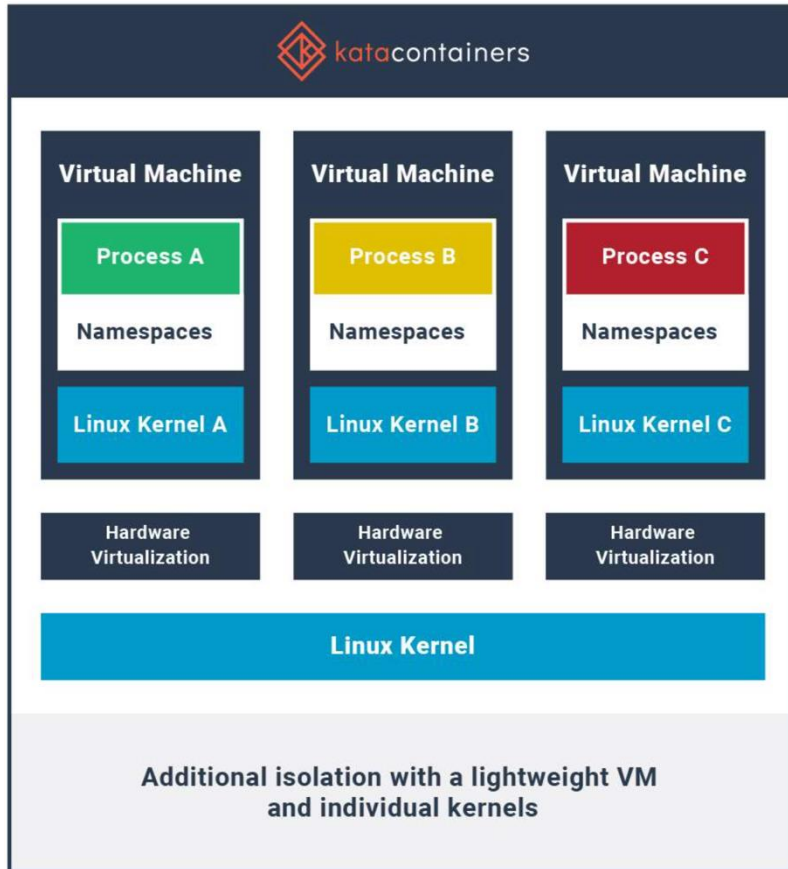
- Yes, shared kernels are more efficient
 - And easily scriptable
- But have weaker isolation boundaries (compared to VMs)
 - Escapes are possible – through regularly discovered vulnerabilities
 - Many high-profile CVEs
<https://github.com/haseeburrehmanfaheem/Container-Escape-CVEs>
 - Kernel failure modes can affect all active containers (e.g., kernel panic)
- Another pivot has occurred
 - New runtimes that enforce stronger boundaries, but keep existing container workflows

How to reinforce containers

- Ideally, we'd like to keep all of the workflows
 - So let's just replace the low-level runtime
- Kata:
 - Runs a proper microvm hypervisor (Firecracker VMM + KVM)
 - Inside, there's a real bare-bones OS with container(s) on top of it
 - Container-like API for a VM + container in VM
- gVisor
 - Runs a special userspace kernel



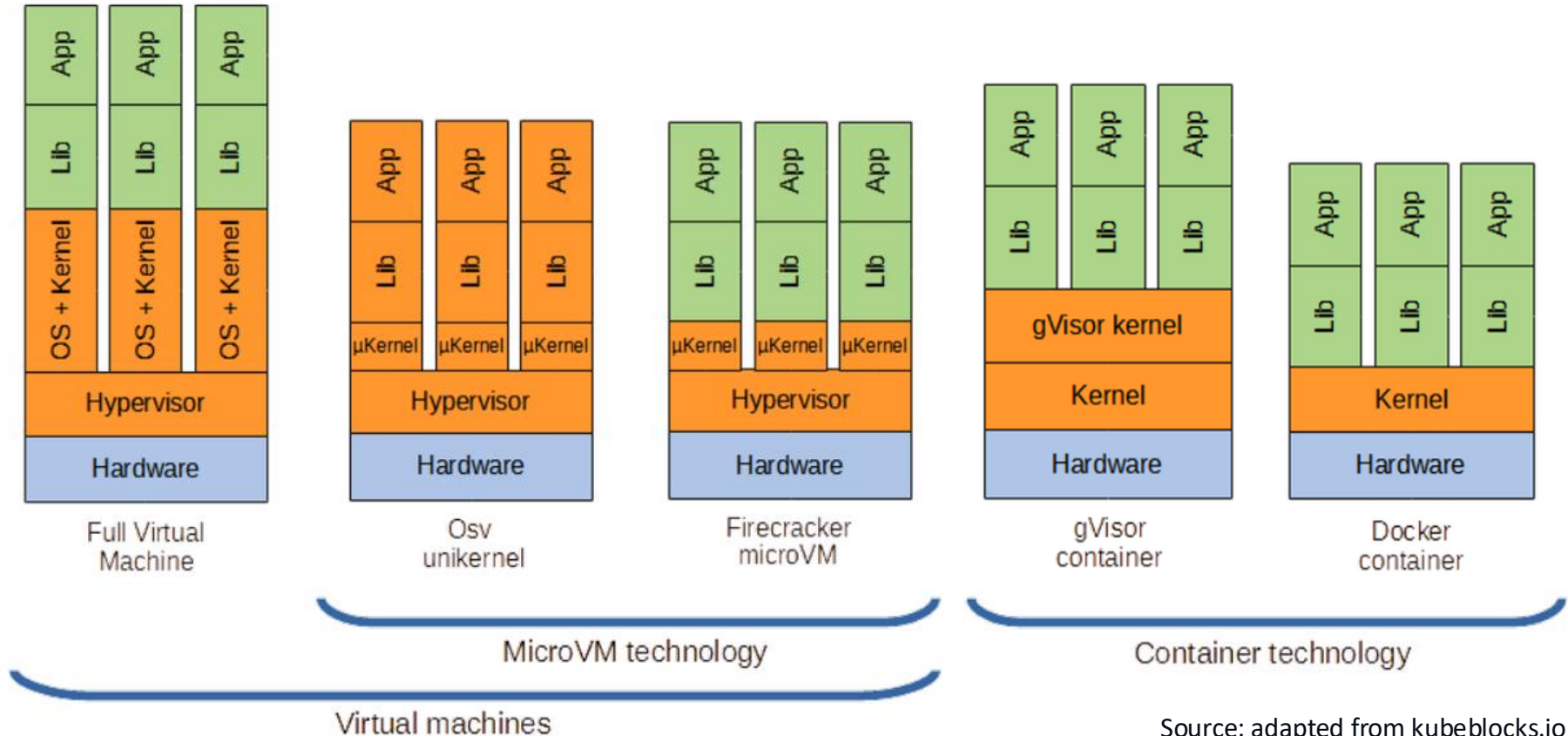
Kata containers



Source: <https://katacontainers.io/learn/>



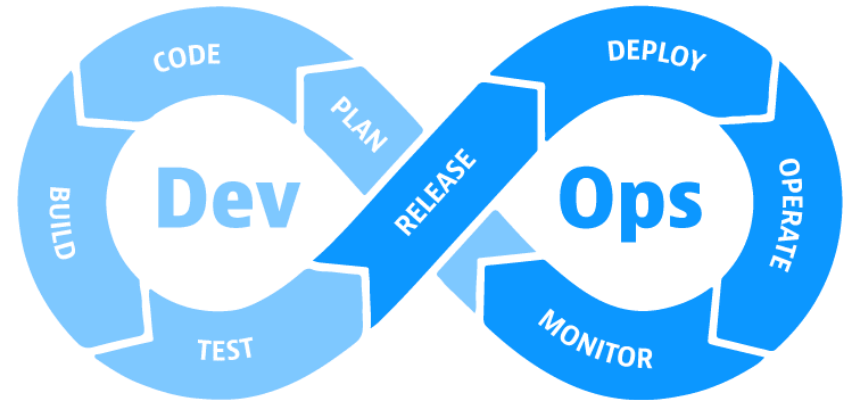
Isolation spectrum



Source: adapted from kubeblocks.io

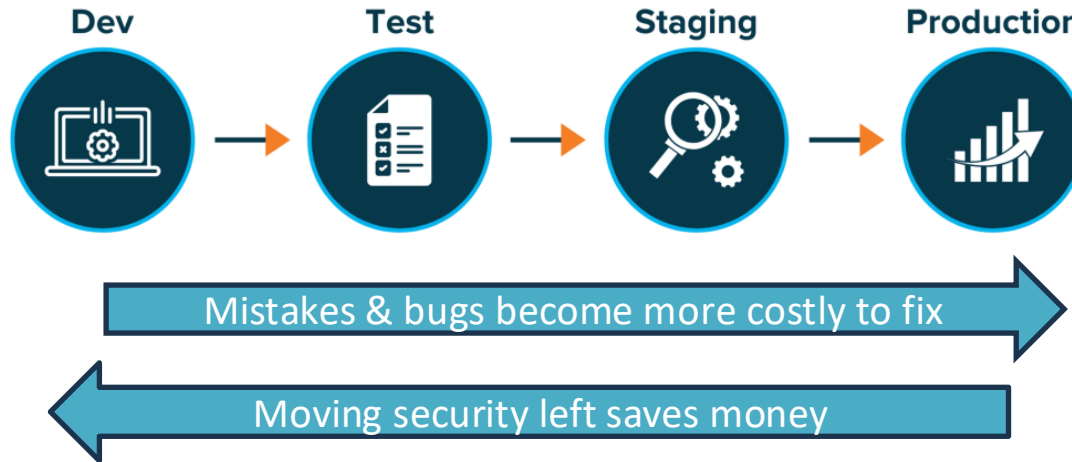
Improved workflows

- Large part of the appeal of containers is the workflow
 - A basic form of "infrastructure as code" → everything becomes a YAML file
 - A way to build an image declaratively
 - A way to compose containers – declaratively
- Perfect fit for the DevOps role
 - Development + operations in a single person
 - Developer that can deploy code and also manage infrastructure(-as-code)
 - Continuous integration / Continuous deployment (CI/CD) uses automation to build test, integrate & deploy code



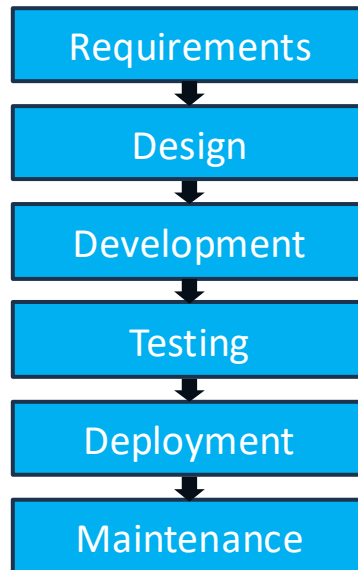
Improved workflows

- Emergence of DevSecOps
 - "Shifting security left"
 - Every phase of the dev and ops must become security-aware



Upgrading SDLC to SSDLC

- Secure Software Development Life-Cycle



#	Secure SDLC phase	Objective	Additional required processes
1	Security requirements	Define relevant security requirements for the software.	Security requirements gathering. Data classification and handling. Misuse case modeling. Risk management.
2	Secure design	Understand and analyze potential threats against the system and design secure software architecture.	Threat modeling. Secure design considerations.
3	Secure development	Ensure that code is developed securely and implement the security controls identified during the design phase.	Identification of common weaknesses. Following security best practices. Version control. Securing build environment and leveraging automated SCA and SAST.
4	Security testing	Validate and verify the functionality and security of the developed software.	Validating security controls, including with automated DAST tests. Proper management of test data.
5	Secure deployment	Ensure a secure environment and configuration for the deployment / release of the software.	Defining acceptance criteria for secure deployment/release. Verification and validation of deployment according to acceptance criteria. Certification & accreditation. Securing the deployment.
6	Secure maintenance	Ensure that the software will continue to function in a reliable, resilient, and recoverable manner.	Incident management. Change management. Disposal.

Some good S-SDLC practices

- Threat modeling, misuse case modeling
- Vulnerability management
- Dependency tracking and Software Composition Analysis
- Static Application Security Testing (SAST)
- Dynamic Application Security Testing (DAST)
- Reliability testing
- All of these can be added to the CI/CD pipeline
 - Automation

